Repaso UT1. Implantación de arquitecturas web

Ciclo de Despliegue de Aplicaciones Web (DAW) · Miguel Ángel Rubiano Barros

Repaso visual de todo lo impartido en el Tema 1: arquitecturas cliente/servidor, servidores web y de aplicaciones, Apache, XAMPP/LAMP, Tomcat y la estructura de una aplicación web.

Apartado 1 · La arquitectura web y algunos modelos

Una arquitectura web es la estructura que permite acceder a una aplicación desde distintas máquinas. Casi siempre sigue el modelo **cliente/servidor**: una comunicación asimétrica en la que un extremo ofrece servicios y el otro los usa.



El navegador pide una página por HTTP; el servidor la localiza y responde. Después la conexión se rompe.

Tres componentes

Un servidor web (Apache), una red con protocolos (Internet bajo TCP/IP) y uno o más clientes (navegadores).

Scripts

Los de cliente (JavaScript) no acceden a los recursos del servidor; la lógica de negocio solo se activa al ejecutarse scripts en el servidor.

Componentes de la app

Lógica de negocio (define los procesos), administración de datos (BD y archivos) e interfaz (navegador, móvil, tablet).

Dos capas frente a tres capas

Dos capas	Tres capas
Capas Cliente y servidor	**Capas** Presentación, proceso y datos
Lógica de negocio En el cliente (*fat client*), dentro de la app que hace la interfaz	**Lógica de negocio** Centralizada en la capa intermedia de proceso (servidor web)
Servidor Solo administra los datos	**Servidor** Web (proceso) y de datos (SGBD, ficheros, XML, correo)
Hoy Modelo superado	**Hoy** Modelo de las aplicaciones web actuales



- **Capa de presentación:** parte en el cliente y parte en el servidor; recoge datos del usuario, los envía a proceso y genera la presentación con el resultado.
- **Capa de datos:** garantiza seguridad e integridad. Con páginas estáticas no existe, porque los datos van en los propios archivos de marcas.
- **Páginas dinámicas** (PHP, Python, Ruby, Java): el servidor necesita un módulo adicional, empotrado en el servidor web (LAMP) o un servidor de aplicaciones aparte (Tomcat para Java).

Apartado 2 · Un modelo simple para el despliegue de aplicaciones web

Una aplicación web es una aplicación distribuida con, al menos, una parte en el servidor y otra en el navegador. Los aspectos clave son **seguridad, velocidad y estabilidad**.

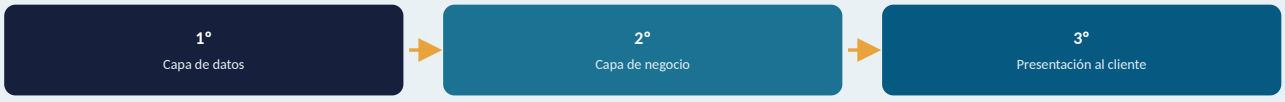
Antes de firmar un contrato o dar un presupuesto hay que valorar: la idea general de la aplicación, las tres capas, no usar siempre las mismas tecnologías y el coste disponible.

Todo proyecto pasa por cuatro fases, con el mantenimiento como fase aparte una vez en producción:



El mantenimiento queda fuera del ciclo: suele ser un contrato aparte (corregir errores, hacer mejoras).

Y conviene construirlo **de abajo a arriba**: primero los datos, luego el negocio, y al final la presentación. Hacerlo al revés indica necesidades mal identificadas y provoca más errores y modificaciones.

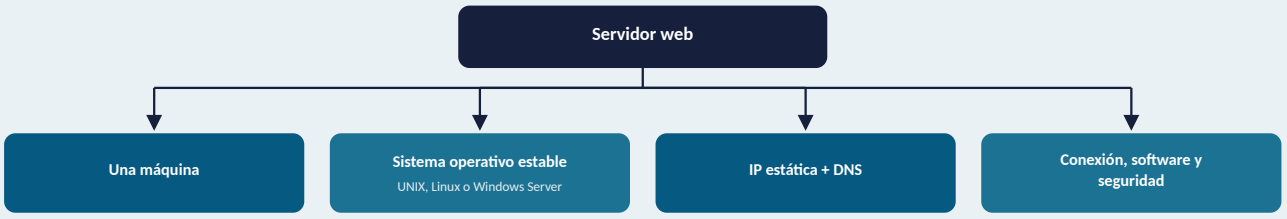


Apartado 3 · Servidores web

Un servidor web es un **programa** (no la máquina) que da un servicio a través de una red, normalmente por HTTP. Una máquina puede alojar varios servidores web o cumplir otras funciones. El más extendido es **Apache**.

Problemas típicos del modelo distribuido: latencia y poca fiabilidad del transporte, falta de memoria compartida, fallos parciales, acceso concurrente a recursos remotos y actualizaciones de alguna de las partes.

Qué se necesita para montar un servidor web



DNS: convierte nombres legibles en direcciones IP y viceversa; una IP dinámica (tipo dyn.com) solo es recomendable para servidores con muy poca carga. Hace falta además conexión 24 horas y una configuración que bloquee el acceso a partes críticas.

Hosting

No hay que comprar ni mantener hardware ni software; la fiabilidad suele ser muy alta. Un servidor propio solo compensa con cargas grandes o si se necesita controlar todo.

Gestores de contenidos

Para sitios cuyo aspecto no cambia pero su contenido sí. Joomla y Drupal son gratuitos; WordPress nació como blog y es sencillo para webs informativas.

Servicios web

Tecnología con protocolos y estándares (SOAP, XML) para intercambiar datos entre aplicaciones a través de HTTP: una colección de procedimientos invocables desde cualquier lugar, independiente de la plataforma y el lenguaje, definida en una API. Es un concepto más ambicioso que el de servidor web.



Apartado 4 · Instalación y configuración básica de un servidor web: Apache

Por qué Apache

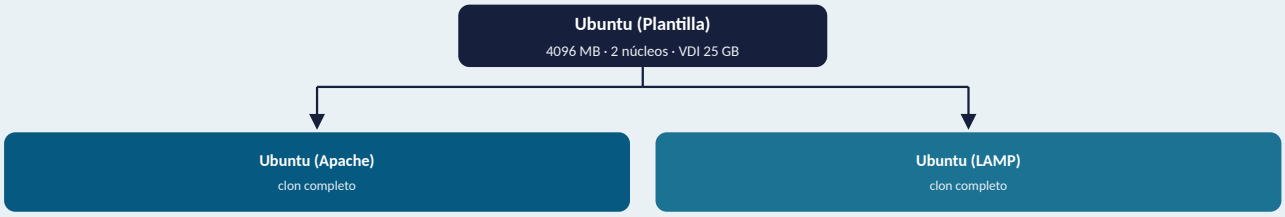
Diseño modular y muy configurable, código abierto, funciona muy bien con PHP y Perl. Hay versiones para Windows, Linux y Mac.

LAMP y XAMPP

Instaladores que dejan Apache, MySQL y PHP funcionando juntos (XAMPP añade Perl). Exigen conocer la configuración de los tres para un servidor seguro.

Entorno de prácticas

VirtualBox con una máquina Ubuntu de escritorio (instalación mínima). De ella se hacen clonaciones completas:



Instalación y comprobación

```
$ sudo apt update          # actualiza los repositorios oficiales
$ sudo apt upgrade         # actualiza el software ya instalado
$ sudo apt install apache2 # instala el servidor
$ apache2 -v               # muestra la versión (Apache/2.4.29)
```

En el navegador, `http://localhost` debe mostrar la página «It works!». Una página propia (por ejemplo `saludo.html`) se coloca en `/var/www/html`.

Configuración centralizada

`/etc/apache2/apache2.conf`, desde el que se llama a otros ficheros más específicos.

Arranque automático

El script `/etc/init.d/apache2` admite start, stop, restart y graceful.

DSO

Dynamic Shared Objects: permite añadir módulos después sin recompilar el servidor.

4.2 Comprobación y eliminación de otras versiones

La versión de escritorio de Ubuntu no trae Apache, MySQL ni PHP. Si hay una instalación previa y se quiere una instalación limpia (solo en situaciones excepcionales):

1. `dpkg --list | grep -e apache -e mysql -e php` — busca paquetes instalados (`ii` = instalado).
2. `ps -ef | grep apache2` y `ps -ef | grep mysql` — buscan procesos en ejecución.
3. `sudo service apache2 stop` y `sudo /etc/init.d/mysql stop` — paran los servidores. PHP no se para: es una herramienta para hacer aplicaciones, no un servidor.
4. `dpkg -l | grep -e apache -e mysql -e php | awk '{print $2}' | xargs sudo dpkg --purge` — desinstala todo.

Apartado 5 · Instalar un servidor con LAMP

LAMP son **L**inux, **A**pache, **M**ySQL y **P**HP (también Perl o Python). Se instala en una máquina nueva con `sudo apt install lamp-server^` y se prueba parte por parte:

Apache

Abrir `localhost` en el navegador.

PHP

Crear `/var/www/html/prueba.php` con `<?php phpinfo(); ?>` y abrir `localhost/prueba.php`.

MySQL

Por consola con `sudo mysql -u root` (salir con `quit`) o con phpMyAdmin.

phpMyAdmin

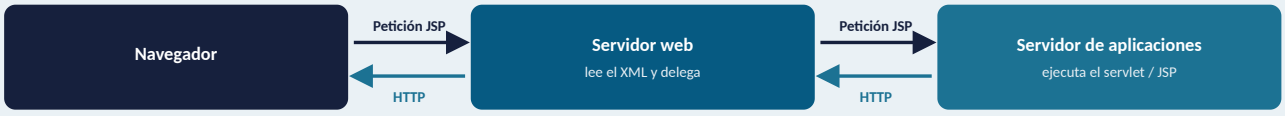
`sudo apt install phpmyadmin`, enlace simólico a `/var/www/html/`, reiniciar Apache y abrir `localhost/phpmyadmin`.

Problema habitual: MySQL se instala con `root` sin contraseña, y phpMyAdmin, por seguridad, no lo permite (aviso «AllowNoPassword»). **Solución:** cambiar la contraseña de `root` o crear un usuario nuevo desde la consola de MySQL con `CREATE USER…`, `GRANT ALL PRIVILEGES ON *.* TO … WITH GRANT OPTION;` y `FLUSH PRIVILEGES;`.

Apartado 6 · Servidores de aplicaciones

Un servidor de aplicaciones es un paquete de software que ofrece a las aplicaciones servicios como **seguridad, servicios de datos, soporte para transacciones, balanceo de carga** y gestión de sistemas distribuidos. El término nació con Java Enterprise Edition y hoy se extiende a otras tecnologías: Tomcat (Java, de Apache), .NET en servidores Microsoft, Zend Server (PHP), Zope (Python), WebLogic y JBoss (Java).

Tomcat está creado para alojar servlets y Java Server Pages (JSP); es gratuito y de código abierto (versión del curso: la 10). El servidor de aplicaciones y el servidor web son dos partes diferenciadas, aunque a veces vengan en un mismo paquete: el primero descarga al segundo de la gestión de ciertos tipos de archivo.



Flujo de una petición JSP: llega al servidor web, que lee un XML del servidor de aplicaciones, decide que lo gestiona este y le envía la petición por HTTP.

Contenedores

Aislan la ejecución de cada aplicación o instancia del resto, dando seguridad, soporte para transacciones, conexión remota y gestión de recursos.

Web

Ejecuta servlets y JSP en un entorno controlado.

EJB

Ejecuta los Enterprise JavaBeans: API para aplicaciones empresariales JEE.

Del cliente de la app

Ejecuta componentes de la aplicación en la máquina cliente.

De applets

Ejecuta applets en el cliente: navegador + plugin de Java.

Servidor J2EE

Programa que proporciona los contenedores EJB y Web.

Los contenedores de **Tomcat** se denominan **Catalina**.

Apartado 7 · Tomcat

Tomcat puede funcionar solo: procesa peticiones HTTP y sirve HTML con bastante eficiencia (aunque peor que Apache). Si se quiere un servidor web con funcionalidad adicional, lo mejor es usar ambos: Apache recibe todas las peticiones y envía a Tomcat lo que le corresponde. En una máquina virtual de Java solo puede ejecutarse una instancia de Tomcat.

7.1 Java

Dos versiones habituales en Linux — la oficial de Oracle y la de código abierto OpenJDK; ambas pueden estar instaladas, pero solo una activa. El **JDK** incluye implícitamente el **JRE**; al revés no (el JRE solo se instala en clientes que no desarrollan en Java).

```
$ sudo apt update
$ sudo apt install default-jre default-jdk
$ java -version        # comprueba la instalación
```

7.2 Instalar Apache Tomcat

Java es obligatorio y debe estar bien configurado. Se instala con `sudo apt install tomcat10`. En la versión 10 se instala como servicio y arranca solo con Ubuntu; la configuración está en `/etc/tomcat10`.

Apache

Puerto **80** — sirve páginas HTTP en general.

Tomcat

Puerto **8080** — `http://localhost:8080`. Página «It works!» estática en `/var/lib/tomcat10/webapps/ROOT/index.html`.

Variables de entorno (en `/etc/environment`, se añaden al PATH y se reinicia con `reboot`):

CATALINA_HOME

`/usr/share/tomcat10` — directorio de instalación (binarios y programa).

CATALINA_BASE

`/var/lib/tomcat10` — directorio de una instancia; distinto para cada una si hay varias.

7.3 Paquetes adicionales (opcionales, muy útiles)

tomcat10-docs

Documentación (en inglés) — `localhost:8080/docs/`

tomcat10-examples

Ejemplos de servlets, JSP y WebSocket — `localhost:8080/examples/`

tomcat10-admin

Apps manager y host-manager — `localhost:8080/manager/html/`

7.4 Usuarios de Tomcat

Para usar sus aplicaciones hay que estar registrado en `/etc/tomcat10/tomcat-users.xml` con usuario y clave. Para entrar en la aplicación **manager** el usuario necesita los roles `manager`, `manager-gui`, `manager-script`, `admin`, `admin-gui` y `admin-script`. Tras editar el fichero hay que reiniciar el servicio.

7.5 Parar, iniciar y reiniciar

```
$ sudo service tomcat10 {stop|start|restart|status}
```

Apartado 8 · Aplicaciones web

Se diferencian de una aplicación estándar en que se accede a ellas a través de una red (Internet o intranet). Se escriben en lenguajes del navegador (JavaScript + HTML) y de los servidores de aplicaciones (Java, PHP…), y necesitan un navegador para mostrarse y ejecutarse.

Ventajas

Distribución
No requieren instalación ni actualización compleja en cada equipo

Cliente
Basta un navegador compatible; no exigen clientes potentes

Integración
Fáciles de integrar con otras funciones del servidor, como el correo

Plataforma
Eliminan problemas de arquitecturas y sistemas operativos

Desventajas

Interfaz
Menos intuitiva y con peor comportamiento que las clásicas

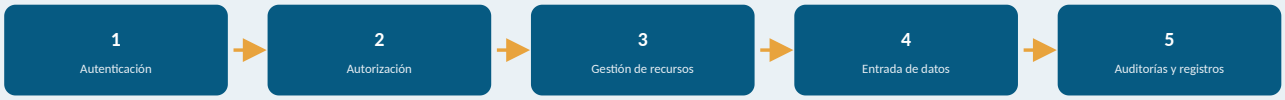
Tecnologías
Muy cambiantes: puede obligar a rehacer la interfaz

Datos
Sin estándares en archivos de oficina, dificulta compartir datos

Red y privacidad
Dependen de la red; preocupa la privacidad y seguridad

Seguridad: cinco áreas

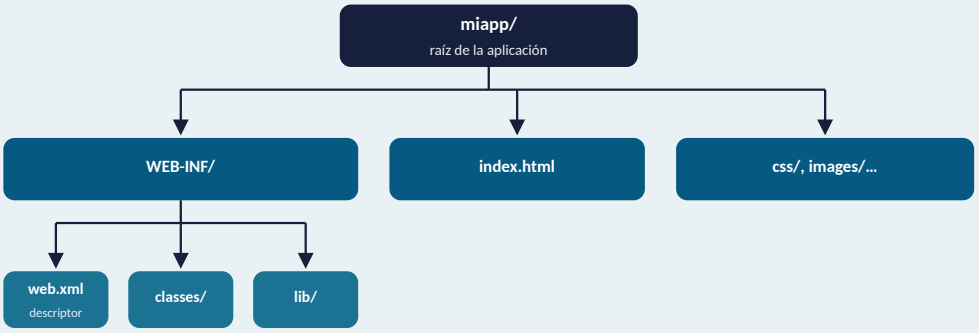
El principal problema es la seguridad, y hay que buscar un compromiso con la eficiencia: demasiada seguridad puede ralentizar el uso y aumentar el tráfico.



Recomendaciones básicas: probar mucho (idealmente con alguien que intente atacar la aplicación) y usar un *framework* común para el equipo, que contemple persistencia de datos, sesiones y autenticación, seguridad, cachés, plantillas e interfaz de administración.

8.1 Estructura

Equivale a la de un servidor web: normalmente en **tres capas** (presentación, aplicación y datos) o vista en dos (cliente y servidor). Según dónde recaiga la mayor parte del trabajo, es de cliente pesado o de servidor pesado. También designa la distribución de directorios:



8.2 Descriptor de despliegue

Es un archivo de configuración del servidor de aplicaciones, en XML en J2EE, que describe cómo desplegar un componente, módulo o aplicación: seguridad, opciones del contenedor y configuración. En una aplicación web J2EE se llama `web.xml` y va en el directorio `WEB-INF` de la raíz de la aplicación. Cada servidor tiene también el suyo (en Tomcat, `/etc/tomcat10/web.xml`); un entorno como Eclipse lo genera, pero conviene revisarlo.

Comandos, rutas y puertos que hay que saber

Qué	Valor
Actualizar repositorios / software instalado	`sudo apt update` / `sudo apt upgrade`
Instalar Apache / ver versión	`sudo apt install apache2` / `apache2 -v`
Configuración principal de Apache	`/etc/apache2/apache2.conf`
Carpeta de las páginas web	`/var/www/html`
Arranque de Apache	`/etc/init.d/apache2` (start, stop, restart, graceful)
Instalar LAMP	`sudo apt install lamp-server^`
Consola de MySQL	`sudo mysql -u root` (salir: `quit`)
Instalar y publicar phpMyAdmin	`sudo apt install phpmyadmin` + enlace simólico en `/var/www/html/`
Instalar Java / comprobar	`sudo apt install default-jre default-jdk` / `java -version`
Instalar Tomcat y extras	`tomcat10` , `tomcat10-docs` , `tomcat10-examples` , `tomcat10-admin`
Variables de Tomcat	`CATALINA_HOME=/usr/share/tomcat10` , `CATALINA_BASE=/var/lib/tomcat10`
Puertos	Apache **80** · Tomcat **8080**
Usuarios de Tomcat	`/etc/tomcat10/tomcat-users.xml`
Configuración / descriptor global de Tomcat	`/etc/tomcat10` / `/etc/tomcat10/web.xml`
Página por defecto de Tomcat	`/var/lib/tomcat10/webapps/ROOT/index.html`
Control del servicio Tomcat	`sudo service tomcat10 {stop|start|restart|status}`
Descriptor de una app J2EE	`web.xml` dentro de `WEB-INF`

Confusiones frecuentes

Servidor web Sirve páginas por HTTP (Apache)	VS	**Servidor de aplicaciones** Ejecuta servlets y JSP, descargando de ese trabajo al servidor web (Tomcat)
Servidor web Es software	VS	**Hardware** Una misma máquina puede alojar varios servidores web
apt update Actualiza los repositorios	VS	**apt upgrade** Actualiza el software ya instalado
CATALINA_HOME Instalación de Tomcat	VS	**CATALINA_BASE** Directorio de cada instancia
Puerto 80 Apache escucha aquí	VS	**Puerto 8080** Tomcat escucha aquí
JDK Incluye el JRE	VS	**JRE** No incluye el JDK; solo se instala en clientes
PHP No se «para»: es una herramienta para hacer aplicaciones	VS	**Un servidor (Apache, MySQL)** Sí se para/arranca como servicio
Dos capas La lógica va en el cliente (fat client)	VS	**Tres capas** La lógica va en la capa de proceso del servidor web
Mantenimiento No es una fase del proyecto; es un contrato aparte	VS	**Las 4 fases (concepto...pruebas)** Sí forman el ciclo del proyecto
Orden de desarrollo De abajo a arriba: datos, negocio, presentación	VS	**Orden «intuitivo»** Empezar por la presentación indica necesidades mal identificadas
Contenedor Web (Tomcat) Ejecuta servlets y JSP	VS	**Contenedor EJB** Ejecuta los Enterprise JavaBeans

Descriptor de despliegue XML llamado web.xml, dentro de WEB-INF	VS	**Catalina** Es el nombre de los contenedores de Tomcat, no el descriptor